

1. Defina as seguintes funções em Haskell:
 - (a) `triple :: Int -> Int` : dado um valor inteiro, o triplica.
 - (b) `maxEven :: Int -> Int` : dado um valor inteiro n , retorna o maior inteiro par mais próximo de n .
 - (c) `isOdd :: Int -> Bool` : dado um valor inteiro n , retorna `True` se, e somente se, n é ímpar.
 - (d) `prod :: Int -> Int -> Int` : dados dois valores inteiros, retorna seu produto.
 - (e) `squared :: Int -> Int` : dado um valor inteiro n , retorna n^2 .
2. Considere um sistema de coordenadas cartesianas expresso como pares de valores inteiros. Em Haskell, podemos representar um ponto por meio de um par de inteiros `(Int,Int)`. Implemente as seguintes funções em Haskell. Para alguns destes exercícios, você pode precisar consultar algum material sobre álgebra linear.
 - (a) `moveX :: Int -> (Int,Int) -> (Int,Int)` dada uma distância inteira d e um ponto $p = (x, y)$, move p d unidades ao longo do eixo X .
 - (b) `moveY :: Int -> (Int,Int) -> (Int,Int)` dada uma distância inteira d e um ponto $p = (x, y)$, move p d unidades ao longo do eixo Y .
 - (c) `mirrorY :: (Int,Int) -> (Int,Int)` dado um ponto, retorna um novo ponto simétrico ao eixo Y .
 - (d) `mirrorX :: (Int,Int) -> (Int,Int)` dado um ponto, retorna um novo ponto simétrico ao eixo X .
 - (e) `higher :: (Int,Int) -> (Int,Int) -> Bool` dados dois pontos p_1 e p_2 respectivamente, retorna `True` se, e somente se, p_1 está estritamente mais alto que p_2 .
 - (f) `distance :: (Int,Int) -> (Int,Int) -> Float` dados dois pontos p_1 e p_2 respectivamente, retorna a distância entre p_1 e p_2 .
 - (g) `colinear :: (Int,Int) -> (Int,Int) -> (Int,Int) -> Bool` dados três pontos p_1 , p_2 e p_3 , retorna `True` se, e somente se, os três pontos são colineares.
3. Use recursão, casamento de padrões e guardas, quando necessário, para resolver cada um dos seguintes exercícios.
 - (a) `quo :: Int -> Int -> Int` dados dois valores inteiros positivos n_1 e n_2 , tais que $n_1 > n_2$, retorna o número de vezes que é possível subtrair n_2 de n_1 sem que o resultado seja menor que zero. Você não pode usar qualquer função pronta de Haskell, exceto subtração e comparações, para resolver o problema.
 - (b) `res :: Int -> Int -> Int` dados dois valores inteiros positivos n_1 e n_2 , tais que $n_1 > n_2$, retorna o valor remanescente quando se subtrai n_2 de n_1 até que não seja mais possível obter um resultado maior ou igual a zero.
 - (c) `mdc :: Int -> Int -> Int` dados dois números inteiros positivos n_1 e n_2 , determina o maior divisor comum entre esses números.
 - (d) `fib :: Int -> Int` dado um número inteiro positivo n , retorna o n -ésimo termo da série de Fibonacci.